Laporan Responsi Praktikum Pemrograman Mobile

Aplikasi Profil Mahasiswa dengan Catatan Kuliah

Disusun Oleh:

Nama : Rangga Pasha Cucu Wibisono

NIM : 25.11.6570

Kelas : S1IF06

Program Studi : S1 Informatika

Mata Kuliah : Pemrograman Mobile (ST407)

Tanggal Responsi : 12 November 2026

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS AMIKOM YOGYAKARTA

2026/2027

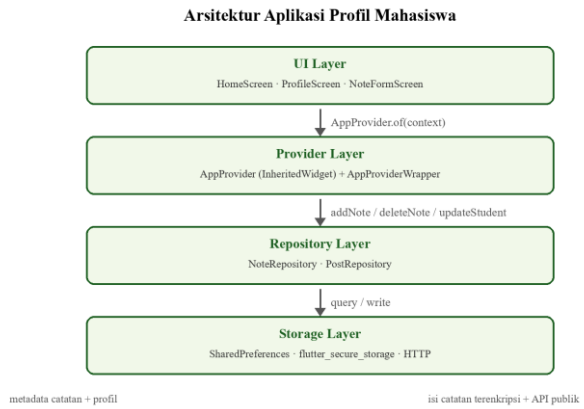
1. Deskripsi Aplikasi

Aplikasi Profil Mahasiswa dengan Catatan Kuliah dibangun menggunakan Flutter dan menerapkan materi Pertemuan 1–6. Terdapat dua fitur utama: profil mahasiswa yang dapat diedit, serta catatan kuliah yang dapat ditambah, dilihat, dan dihapus dengan validasi input dan penyimpanan lokal yang aman. Sebagai pelengkap, disajikan tab Info Kampus yang mengambil data dari API publik.

- Profil Mahasiswa: nama, NIM, program studi, dan email ditampilkan pada kartu profil dengan avatar inisial, serta dapat diubah melalui formulir tervalidasi.
- Catatan Kuliah: judul dan isi divalidasi lalu disanitasi sebelum disimpan; metadata dan isi catatan dipisahkan ke dua media penyimpanan berbeda.
- Info Kampus: daftar data dari endpoint /posts dilengkapi state loading, error, dan aksi coba lagi.
- Aplikasi dijalankan pada target Windows desktop (flutter run -d windows) agar dapat diuji tanpa perangkat Android.

2. Arsitektur Aplikasi

Aplikasi memisahkan tanggung jawab menjadi empat lapisan. UI hanya membaca state dan memanggil aksi; seluruh akses data melewati repository; penyimpanan fisik berada di lapisan paling bawah.



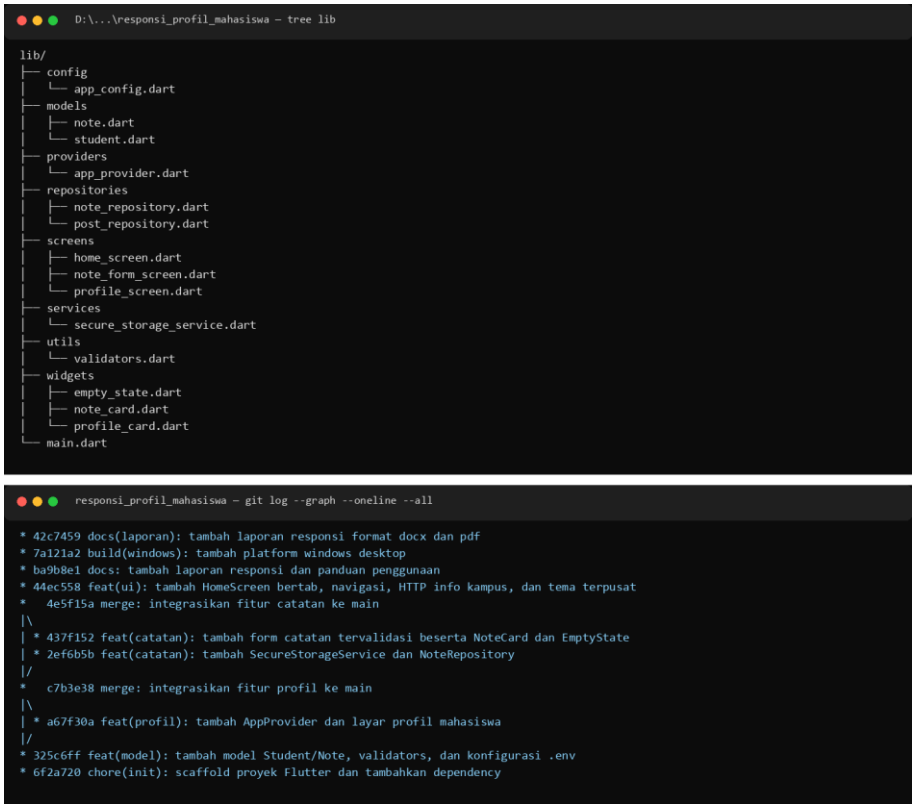
Gambar 1. Arsitektur aplikasi (UI → Provider → Repository → Storage)

Alur data berjalan satu arah. Widget memanggil aksi pada AppProvider, wrapper memperbarui state, repository menulis ke SharedPreferences, flutter_secure_storage, atau HTTP, lalu hasilnya dikembalikan sehingga UI dirender ulang.

3. Langkah Kerja

3.1 Persiapan Proyek, Struktur Direktori, dan Git

1. Membuat proyek dengan perintah flutter create --project-name responsi_profil_mahasiswa --org com.amikom.
2. Menyusun direktori lib/ menjadi config/, models/, repositories/, providers/, screens/, services/, utils/, dan widgets/ sesuai spesifikasi soal.
3. Menambahkan dependensi shared_preferences, flutter_secure_storage, flutter_dotenv, http, dan intl.
4. Menginisialisasi Git, membuat branch fitur-profil dan fitur-catatan, lalu melakukan merge --no-ff ke main.
5. Menambahkan .gitignore untuk .env, build/, dan .dart_tool/, serta menyediakan .env.example sebagai template.



Gambar 2. Struktur direktori lib/ dan riwayat commit git log --graph --oneline --all

3.2 Model, Validators, dan AppConfig

Model Student dan Note dibuat pada direktori models/. Note memisahkan metadata dari isi catatan: method toMetadata() sengaja tidak menyertakan content agar teks catatan tidak

pernah masuk ke SharedPreferences. Validators memusatkan aturan validasi dan sanitasi, sedangkan AppConfig menjadi satu-satunya pintu pembacaan konfigurasi.

```
static String? validateTitle(String? value) {
  final text = (value ?? '').trim();
  if (text.isEmpty) return 'Judul catatan wajib diisi';
  if (text.length < 3) return 'Judul minimal 3 karakter';
  if (text.length > 50) return 'Judul maksimal 50 karakter';
  return null;
}
```

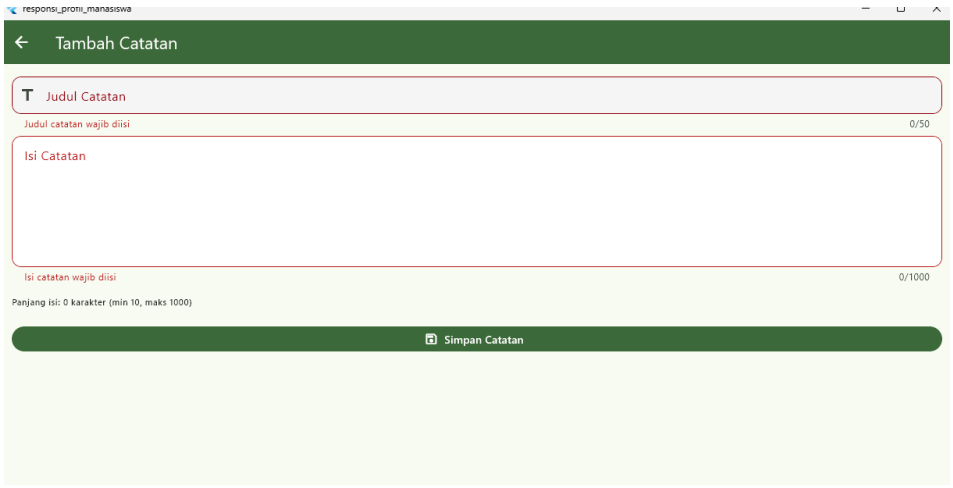
3.3 State Management dengan InheritedWidget

AppProvider merupakan turunan InheritedWidget yang menyediakan student, notes, updateStudent, addNote, dan deleteNote. State sebenarnya dikelola AppProviderWrapper (StatefulWidget) yang memuat data awal pada initState lalu meneruskannya ke subtree.

```
static AppProvider of(BuildContext context) {
  final provider = context.dependOnInheritedWidgetOfExactType<AppProvider>();
  assert(provider != null, 'AppProvider tidak ditemukan pada context ini.');
  return provider!;
}
```

3.4 Form Catatan dan Validasi Input

1. Membuat NoteFormScreen dengan Form, GlobalKey<FormState>, dan dua TextFormField (judul dan isi).
2. Menerapkan validateTitle (3–50 karakter), validateContent (10–1000 karakter), serta sanitize() untuk karakter &, <, >, ", dan '.
3. Menambahkan indikator panjang isi catatan secara langsung agar pengguna mendapat umpan balik sebelum menyimpan.



Gambar 3. Form catatan menampilkan pesan error validasi

3.5 Penyimpanan Lokal yang Aman

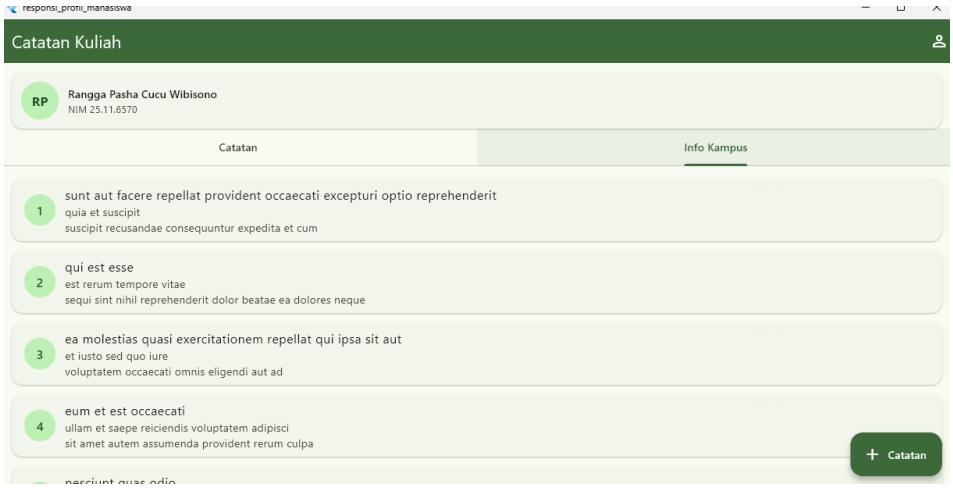
SecureStorageService membungkus flutter_secure_storage dengan kunci note_content_<id>. NoteRepository menyimpan metadata sebagai JSON list pada SharedPreferences dan isi catatan pada secure storage, sehingga keduanya tidak pernah tercampur.

```
Future<void> saveContent(String noteId, String content) {
  return _storage.write(key: 'note_content_$noteId', value: content);
}
```

Tidak ada kredensial hardcoded: BASE_URL dan API_KEY dibaca dari .env yang diabaikan Git. Logging hanya mencetak endpoint dan status HTTP, tidak pernah isi catatan atau API key. AndroidManifest.xml memuat permission INTERNET dan android:allowBackup="false".

3.6 HTTP Dasar dan Penanganan Error

PostRepository mengambil data GET {BASE_URL}/posts dengan timeout 10 detik. SocketException, TimeoutException, ClientException, dan FormatException dikonversi menjadi PostException berisi pesan ramah pengguna, lalu ditampilkan pada EmptyState beserta tombol coba lagi.

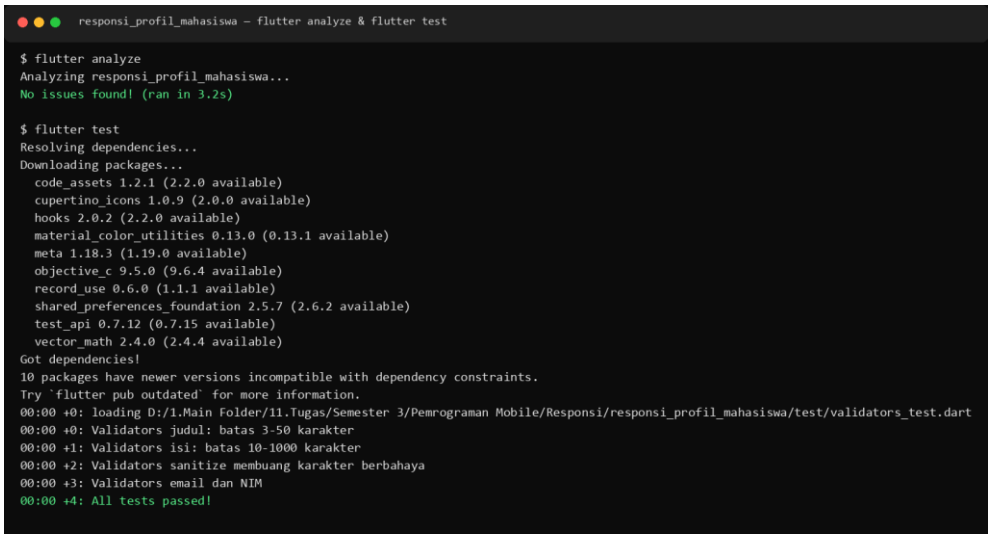


Gambar 4. Tab Info Kampus menampilkan data hasil request HTTP GET

3.7 Integrasi UI, Navigasi, dan Pengujian

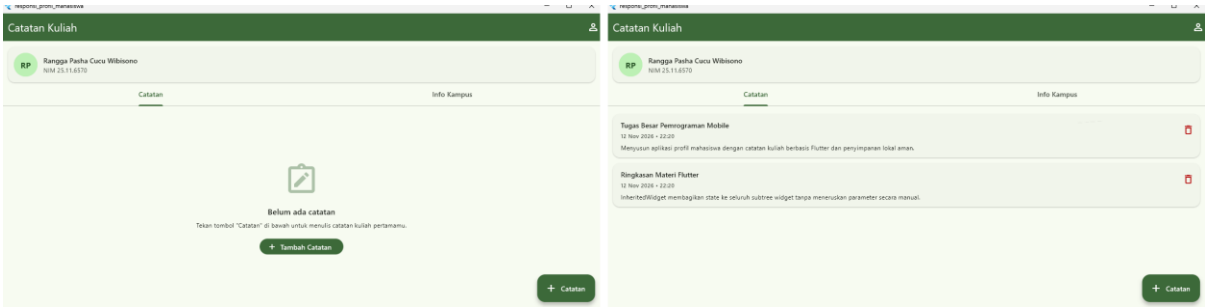
1. HomeScreen menampilkan profil singkat, TabBar (Catatan dan Info Kampus), ListView.builder berisi NoteCard, EmptyState saat daftar kosong, serta FloatingActionButton.

2. Navigasi memakai Navigator.push dengan return value: NoteFormScreen mengembalikan true bila penyimpanan berhasil, dan dialog hapus mengembalikan nilai konfirmasi.
3. Tema dipusatkan pada ThemeData di main.dart, termasuk AppBarTheme, CardThemeData, dan InputDecorationTheme.
4. Menjalankan flutter analyze (No issues found!) dan flutter test (4 pengujian Validators lulus).

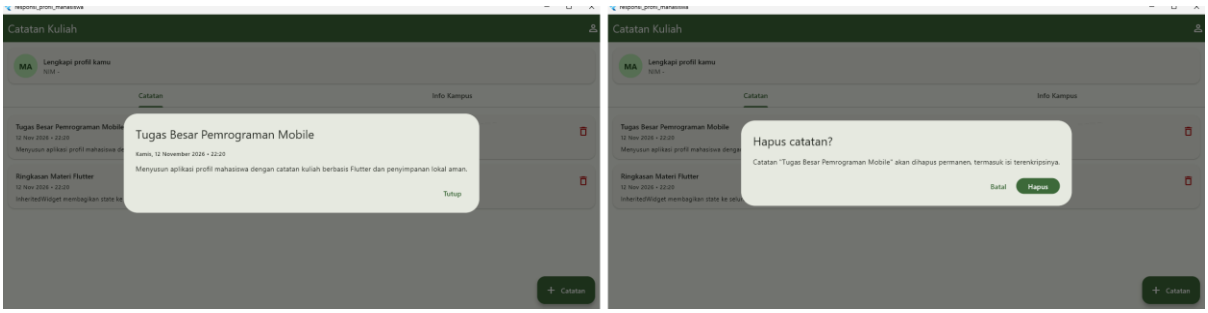


Gambar 5. Hasil flutter analyze dan flutter test

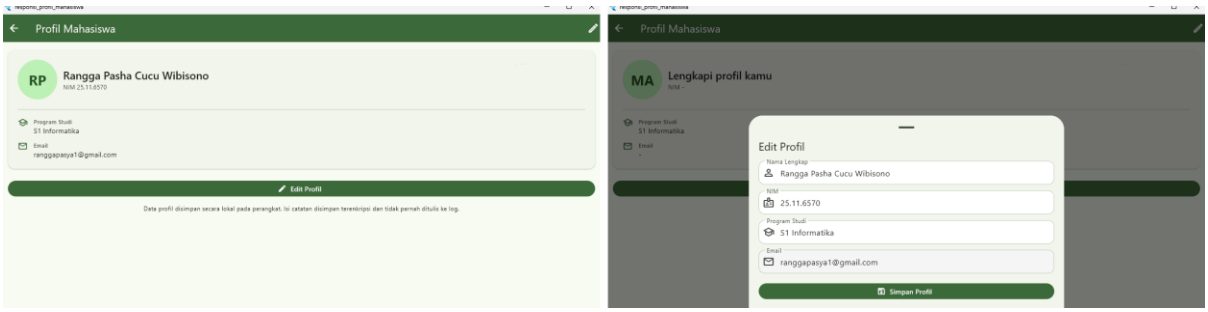
4. Tampilan Aplikasi



Gambar 6. HomeScreen: keadaan kosong (kiri) dan daftar catatan (kanan)



Gambar 7. Dialog detail catatan (kiri) dan dialog konfirmasi hapus (kanan)



Gambar 8. ProfileScreen (kiri) dan bottom sheet edit profil (kanan)

5. Penerapan Sub-CPMK

5.1 sub.cpmk.ST407.CPMK051.1 — Instalasi, Struktur Proyek, dan Git

- Proyek dibuat sesuai nama yang diminta, dengan struktur direktori wajib lengkap.
- Terdapat lebih dari 10 commit dan 2 branch fitur yang di-merge ke main (bukti pada Gambar 2).
- .gitignore mengabaikan .env, build/, dan .dart_tool/.

5.2 sub.cpmk.ST407.CPMK051.2 — Widget Modular dan Separation of Concerns

- ProfileCard dipisahkan dari ProfileScreen dan dipakai ulang pada header beranda dalam mode compact.
- Layout memakai Row, Column, Card/Container, dan EdgeInsets secara konsisten.
- Logika data tidak berada pada widget; seluruh akses melewati provider dan repository.

5.3 sub.cpmk.ST407.CPMK051.3 — State Management dan Validasi

- AppProvider (InheritedWidget) menyediakan data dan aksi; AppProviderWrapper mengelola state.
- updateShouldNotify hanya memberi tahu ketika student, notes, atau isLoading berubah.
- Seluruh input divalidasi dengan aturan panjang eksplisit dan disanitasi sebelum disimpan.

5.4 sub.cpmk.ST407.CPMK051.4 — Keamanan Penyimpanan Lokal dan HTTP

- Isi catatan terenkripsi pada flutter_secure_storage; metadata pada SharedPreferences.
- Konfigurasi berasal dari .env; tidak ada kredensial pada kode maupun riwayat Git.
- Request HTTP memiliki timeout dan penanganan error; log tidak memuat data sensitif.

6. Kendala dan Solusi

- Build APK gagal karena Android Gradle Plugin meminta NDK 28.2.13676358, sementara kapasitas drive C: tidak mencukupi untuk mengunduh dan mengekstrak NDK. Solusi: aplikasi dijalankan pada target Windows desktop (flutter run -d windows) dan NDK dipasang terpisah ketika ruang disk tersedia.
- Berkas .env diabaikan Git sehingga asset hilang saat proyek di-clone. Solusi: disediakan .env.example dan langkah copy .env.example .env pada README.
- BuildContext dipakai setelah await sehingga memicu peringatan use_build_context_synchronously. Solusi: provider dan ScaffoldMessenger diambil sebelum await, lalu dijaga dengan pengecekan mounted.
- Tanggal berbahasa Indonesia memicu LocaleDataException (diselesaikan dengan initializeDateFormatting('id_ID')) dan widget test bawaan flutter create masih menguji aplikasi counter (diganti pengujian Validators).

7. Kesimpulan

1. Struktur proyek yang modular dan penggunaan Git dengan branch fitur memudahkan penelusuran perubahan setiap Sub-CPMK.
2. InheritedWidget cukup untuk mengelola state global sederhana selama state tunggal dikelola satu wrapper dan diakses lewat method of(context).
3. Pemisahan metadata dan isi catatan pada dua media penyimpanan menutup kebocoran teks catatan ke preferensi biasa.
4. Validasi dan sanitasi berlapis mencegah data tidak valid maupun karakter berbahaya masuk ke penyimpanan.
5. Penanganan error HTTP yang eksplisit membuat kegagalan jaringan tampil sebagai pesan yang dapat ditindaklanjuti pengguna.

8. Link Repositori

https://github.com/ranggapasha/responsi-pemrograman-mobile-25.11.6570